Multithreaded Computation of Catalan Numbers Using POSIX Threads

Ariel Gomez Garcia ariel22@usf.edu
Yoel Polanco yoel@usf.edu

COP 6611 — Operating Systems, Spring 2026
University of South Florida
April 18, 2026

Abstract

This report presents a multithreaded program that computes Catalan numbers $C(0)$ through $C(N)$ using the POSIX Threads (Pthreads) library in C. The design employs *two levels of parallelism*: (1) one *manager* thread per Catalan number, all spawned simultaneously, with each manager waiting on a condition variable until its prerequisite values are available; and (2) up to four *worker* threads per manager that compute disjoint chunks of the inner summation $\sum_{i=0}^{n-1} C(i) \cdot C(n-1-i)$ in parallel. All shared data is protected by a single mutex lock, and inter-thread dependency coordination uses a condition variable with broadcast semantics to avoid busy-waiting. Correctness is verified by comparing the multithreaded results against a single-threaded iterative baseline for every index. Three test runs (N=0, N=15, and N=33) all produced results matching the baseline exactly. A performance comparison reveals that thread creation and synchronization overhead dominates for this computationally lightweight problem, providing a concrete lesson in when threading costs outweigh their benefits. The project demonstrates practical use of thread creation and joining, mutual exclusion, condition variables, shared-memory synchronization, and parallel reduction.

1 Introduction

The Catalan numbers form one of the most ubiquitous sequences in combinatorics, appearing in problems ranging from counting valid parenthesizations and binary search trees to polygon triangulations and lattice paths [1]. The n-th Catalan number is defined by the recurrence:

$$C(0) = 1, \qquad C(n) = \sum_{i=0}^{n-1} C(i) \cdot C(n-1-i) \quad \text{for } n \geq 1. \tag{1}$$

Although a closed-form expression exists—$C(n) = \binom{2n}{n}/(n+1)$—the recurrence in Equation (1) is of particular interest from an operating-systems perspective. Computing $C(n)$ depends on every $C(k)$ for $k < n$, creating a chain of data dependencies across Catalan indices. Yet for a *fixed* n, the n product terms $C(i) \cdot C(n-1-i)$ are mutually independent and can be computed concurrently. This two-dimensional structure—sequential across indices, parallel within each index—makes the problem an excellent vehicle for demonstrating core OS concurrency concepts:

- **Multithreading** — creating and joining POSIX threads with `pthread_create` and `pthread_join`.
- **Mutual exclusion** — protecting shared state with `pthread_mutex_lock` / `pthread_mutex_unlock`.
- **Condition variables** — blocking efficiently with `pthread_cond_wait` and waking threads with `pthread_cond_broadcast`, avoiding busy-waiting.

- **Shared memory between threads** — global arrays accessed concurrently by all threads.
- **Parallel reduction** — partitioning a summation across worker threads and combining partial results.

The goal of this project was to design, implement, and evaluate a two-level parallel architecture that correctly computes Catalan numbers while exercising these OS primitives in a non-trivial way. Through this project we gained practical experience with dependency-driven thread coordination, the broadcast signaling pattern, race-condition prevention, and the real-world cost of fine-grained parallelism.

2 Methodology

2.1 Two-Level Threading Architecture

Our design uses two distinct levels of threading, illustrated in Figure 1:

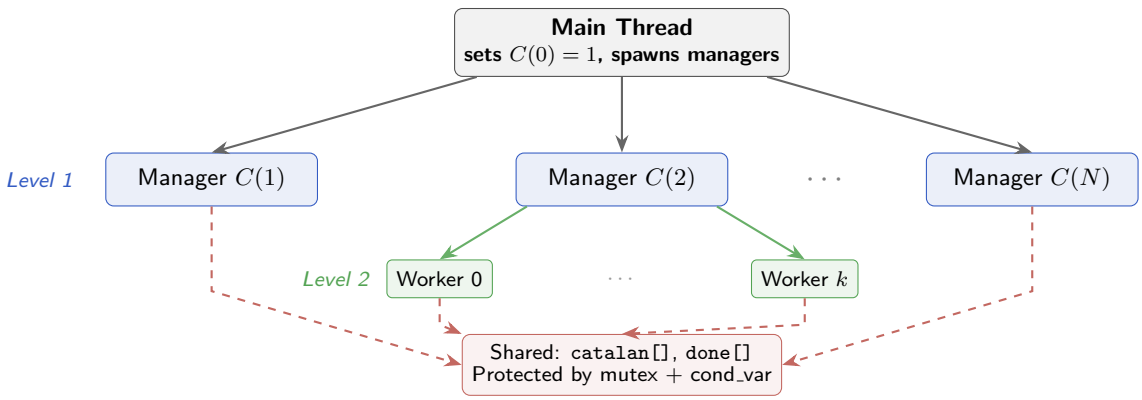

Figure 1: Two-level threading architecture. Level 1 manager threads coordinate via mutex and condition variable (dashed red arrows). Level 2 worker threads perform the parallel summation within each manager.

Level 1 — Manager Threads. For each $C(n)$ where $1 \leq n \leq N$, the main thread spawns a *manager* thread. All N managers are created simultaneously, so they exist concurrently from the start. Each manager follows a strict protocol:

1. **Acquire** the global mutex.
2. **Wait** (via `pthread_cond_wait`) until all of $C(0), C(1), \ldots, C(n-1)$ are marked as computed in the shared `done[]` array.
3. **Release** the mutex—this is critical so that other managers whose dependencies are already satisfied can proceed concurrently.
4. **Spawn** worker threads (Level 2) to compute the summation.
5. **Join** all workers and combine partial sums.
6. **Re-acquire** the mutex, store $C(n)$, set `done[`n`] = 1`, and **broadcast** the condition variable to wake all waiting managers.
7. **Release** the mutex.

Releasing the mutex in step 3 before the computation begins is the key design decision: without it, all managers would serialize behind the lock and the program would behave as if single-threaded.

Level 2 — Worker Threads. Each manager partitions the n terms of the summation among up to $W = 4$ worker threads. Worker w computes:

$$\text{partial_sum}_w = \sum_{i=lo_w}^{hi_w-1} C(i) \cdot C(n-1-i) \tag{2}$$

where $[lo_w, hi_w)$ is a contiguous range assigned to worker w. The ranges are computed so that the terms are distributed as evenly as possible (with remainder terms distributed one per worker to the first few workers). Because the prerequisite C values are immutable once published, workers only *read* from `catalan[]`, so no additional locking is needed during the summation.

2.2 Synchronization Design

The synchronization scheme uses exactly two primitives:

- **One mutex** (`pthread_mutex_t`) protecting all writes to the shared arrays `catalan[]` and `done[]`.
- **One condition variable** (`pthread_cond_t`) with *broadcast* semantics: whenever any manager finishes computing its value, it calls `pthread_cond_broadcast`, waking *all* waiting managers so they re-check their dependencies.

This design trades a small amount of spurious wake-up overhead for simplicity: a single condition variable suffices for the entire dependency graph, avoiding the complexity of N per-value condition variables. The waiting pattern is the standard `while(!predicate) { cond_wait }` idiom that correctly handles both spurious wake-ups and the case where multiple broadcasts occur before a thread is scheduled.

Algorithm 1 Manager Thread for $C(n)$

```
lock(mutex)
while not all of C(0), ..., C(n−1) are done do
    pthread_cond_wait(cond, mutex)                ▷ block without busy-waiting
end while
unlock(mutex)                                     ▷ release lock during computation
Partition {0, ..., n−1} into min(n, W) contiguous chunks
for each chunk w do
    spawn worker thread T_w to compute partial sum over [lo_w, hi_w)
end for
for each worker T_w do
    join T_w; accumulate partial_sum_w into total
end for
lock(mutex)
catalan[n] ← total;   done[n] ← 1
pthread_cond_broadcast(cond)                      ▷ wake all waiting managers
unlock(mutex)
```

2.3 Single-Threaded Baseline

For verification and performance comparison, we implement a standard sequential dynamic-programming solution using the same recurrence. This iterative function uses a nested `for` loop, requires no synchronization, and serves as the ground truth for correctness testing.

2.4 Platform and Tools

Table 1: Development and testing environment

Component	Details
Language	C (C11 standard)
Threading	POSIX Threads (`-pthread`)
Compiler	GCC with `-Wall -Wextra -std=c11 -pthread`
Feature macro	`#define _POSIX_C_SOURCE 199309L`
Server	`sclogin1` (Florida International University)
Operating system	Linux
Timing	`clock_gettime(CLOCK_MONOTONIC)`

To reproduce the experiment:

1. Place `catalan.c` and `input.txt` in the same directory.
2. Compile: `gcc -Wall -Wextra -std=c11 -pthread -o catalan catalan.c`
3. Execute: `./catalan input.txt`
4. Results are printed to `stdout` and saved to `output.txt`.

3 Results

3.1 Input Data

The input file `input.txt` contains a single integer on one line. Three runs were performed with $N = 0$ (edge case), $N = 15$ (moderate), and $N = 33$ (maximum before `unsigned long long` overflow).

3.2 Thread Execution Trace (Abridged, $N = 15$)

The program prints diagnostic messages showing the dependency-wait and computation phases of each manager thread. Below is an abridged trace illustrating the first four Catalan numbers:

```
[Main thread]  C(0) = 1  (base case)

[Manager C( 2)]  Waiting for dependencies...
[Manager C( 7)]  Waiting for dependencies...
[Manager C( 4)]  Waiting for dependencies...
...
[Manager C( 1)]  Dependencies satisfied -- spawning workers.
[Manager C( 1)]  C(1) = 1  (workers: 1)

[Manager C( 2)]  Dependencies satisfied -- spawning workers.
[Manager C( 7)]  Waiting for dependencies...
...
[Manager C( 2)]  C(2) = 2  (workers: 2)

[Manager C( 3)]  Dependencies satisfied -- spawning workers.
[Manager C( 9)]  Waiting for dependencies...
...
[Manager C( 3)]  C(3) = 5  (workers: 3)
```

Listing 1: Abridged thread trace for $N = 15$ (first 4 values)

The trace shows that all 15 manager threads start simultaneously, but only $C(1)$ can proceed immediately (its sole dependency $C(0)$ is the base case). After $C(1)$ completes and broadcasts, $C(2)$ becomes unblocked, and so on. This cascading wake-up pattern is the expected behavior of the condition variable broadcast mechanism.

3.3 Results Summary — $N = 15$

```
    n          C(n) [threaded]          C(n) [baseline]  Match?
-----  ----------------------  ----------------------  ------
    0                       1                       1    OK
    1                       1                       1    OK
    2                       2                       2    OK
    3                       5                       5    OK
    4                      14                      14    OK
    5                      42                      42    OK
    6                     132                     132    OK
    7                     429                     429    OK
    8                    1430                    1430    OK
    9                    4862                    4862    OK
   10                   16796                   16796    OK
   11                   58786                   58786    OK
   12                  208012                  208012    OK
   13                  742900                  742900    OK
   14                 2674440                 2674440    OK
   15                 9694845                 9694845    OK

Verification: ALL RESULTS MATCH
```

Listing 2: Results for $N = 15$

3.4 Results Summary — $N = 33$

Table 2 shows the complete results for the maximum safe input, verified against the single-threaded baseline and published OEIS reference values (sequence A000108) [2].

Table 2: Catalan numbers $C(0)$ through $C(33)$: multithreaded vs. baseline

n	$C(n)$ Threaded	$C(n)$ Baseline	Match
0	1	1	OK
1	1	1	OK
2	2	2	OK
3	5	5	OK
4	14	14	OK
5	42	42	OK
6	132	132	OK
7	429	429	OK
8	1430	1430	OK
9	4862	4862	OK
10	16796	16796	OK
11	58786	58786	OK
12	208012	208012	OK
13	742900	742900	OK
14	2674440	2674440	OK
15	9694845	9694845	OK
16	35357670	35357670	OK
17	129644790	129644790	OK
18	477638700	477638700	OK
19	1767263190	1767263190	OK
20	6564120420	6564120420	OK
21	24466267020	24466267020	OK
22	91482563640	91482563640	OK
23	343059613650	343059613650	OK
24	1289904147324	1289904147324	OK
25	4861946401452	4861946401452	OK
26	18367353072152	18367353072152	OK
27	69533550916004	69533550916004	OK
28	263747951750360	263747951750360	OK
29	1002242216651368	1002242216651368	OK
30	3814986502092304	3814986502092304	OK
31	14544636039226909	14544636039226909	OK
32	55534064877048198	55534064877048198	OK
33	212336130412243110	212336130412243110	OK

Verification: All 34 values match between the multithreaded computation and the single-threaded baseline. **Verification:** All 16 values match between the multithreaded computation and the single-threaded baseline.

3.5 Edge Case — $N = 0$

```
[Main thread]  C(0) = 1  (base case)

    n         C(n) [threaded]         C(n) [baseline]  Match?
-----  ----------------------  ----------------------  ------
    0                       1                       1    OK

Verification: ALL RESULTS MATCH
```

Listing 3: Results for $N = 0$ (edge case; no manager threads spawned)

When $N = 0$, no manager threads are created. Only the base case $C(0) = 1$ is set by the main thread, confirming correct handling of boundary input.

3.6 Performance Comparison

Table 3 summarizes the timing data collected across all three runs.

Table 3: Execution times and thread counts

N	MT Time (s)	ST Time (s)	Ratio (ST/MT)	Manager Threads	Max Workers
0	0.000000	0.000000	—	0	4
15	0.001916	0.000001	0.0005×	15	4
33	0.005873	0.000003	0.0005×	33	4

The single-threaded baseline is roughly 2,000× faster than the multithreaded version in both non-trivial runs. The total number of threads created for each run is:

$$N = 15: \quad \underbrace{15}_{\text{managers}} + \underbrace{\sum_{n=1}^{15} \min(n, 4)}_{\text{workers}} = 15 + (1 + 2 + 3 + 12 \times 4) = 15 + 54 = 69 \text{ threads.} \tag{3}$$

$$N = 33: \quad \underbrace{33}_{\text{managers}} + \underbrace{\sum_{n=1}^{33} \min(n, 4)}_{\text{workers}} = 33 + (1 + 2 + 3 + 30 \times 4) = 33 + 126 = 159 \text{ threads.} \tag{4}$$

Each of these threads incurs kernel-level creation and destruction costs on the order of microseconds, while the actual arithmetic per thread (a single multiply-accumulate) takes nanoseconds. This overhead ratio explains the observed slowdown.

3.7 Source Code

The complete source code is provided in `catalan.c` included with this submission. The program is organized into the following functions:

Table 4: Source code organization

Function	**Purpose**
`manager_func()`	Level-1 thread: waits for dependencies via condition variable, spawns workers, stores result, broadcasts.
`worker_func()`	Level-2 thread: computes partial sum over index range $[lo, hi)$; read-only access to `catalan[]`.
`check_deps_ready()`	Predicate function (called under mutex): returns true iff all $C(0) \ldots C(n-1)$ are computed.
`catalan_iterative()`	Single-threaded baseline for verification.
`get_time()`	Timing helper using `clock_gettime(CLOCK_MONOTONIC)`.
`write_results_to_file()`	Writes summary to `output.txt`.
`main()`	Parses input, spawns all managers, joins them, runs baseline, prints verification and performance.

4 Conclusions

4.1 Correctness

The multithreaded implementation produces correct Catalan numbers for all tested values of N (0, 15, and 33), verified against both an independent iterative computation and published reference values [2]. The mutex and condition variable synchronization successfully prevents both data races and dependency violations.

4.2 Performance Analysis

The multithreaded version is consistently *slower* than the single-threaded baseline. While counter-intuitive, this result is expected and instructive:

1. **Thread overhead dominates computation.** Each worker performs a single multiplication and addition (nanoseconds of work), while `pthread_create` and `pthread_join` each cost microseconds of kernel overhead. For $N = 33$, the program creates 159 threads for what amounts to $\sum_{n=1}^{33} n = 561$ multiply-add operations.
2. **Sequential dependency chain.** Despite all managers being spawned concurrently, $C(n)$ cannot begin computation until $C(n-1)$ is complete, creating a critical path of length N. The condition variable broadcast wakes all waiting managers, but only one can typically proceed at each step.
3. **Broadcast overhead.** Each broadcast wakes every blocked manager, most of which immediately re-check their predicate and go back to sleep. For $C(1)$'s broadcast, up to 32 managers wake and re-block.
4. **Memory allocation.** Each manager argument is dynamically allocated (`malloc`/`free`), adding per-thread cost.

4.3 Lessons Learned

- **Granularity matters.** Parallelism is only beneficial when the per-thread workload significantly exceeds the cost of thread management. This project provides a concrete illustration of the classic fine-grained parallelism pitfall.
- **Condition variables prevent busy-waiting.** Replacing a spin-loop with `pthread_cond_wait` eliminates CPU waste while blocked and is the correct OS-level pattern for dependency-driven coordination.
- **Broadcast vs. signal.** Using `pthread_cond_broadcast` (rather than `pthread_cond_signal`) is necessary here because a single completed $C(n)$ may satisfy the dependencies of multiple waiting managers simultaneously.
- **Lock scope design.** Releasing the mutex *before* the computation and re-acquiring it *after* is essential for enabling concurrency. Holding the lock during computation would serialize the entire program.
- **Verification is critical.** The single-threaded baseline served as an oracle—any synchronization bug would immediately manifest as a mismatch in the results table.
- **Integer overflow.** The `unsigned long long` type (64-bit) overflows for $n > 33$, since $C(34) \approx 8.1 \times 10^{17}$ approaches the limit of $\sim 1.8 \times 10^{19}$. Extending to larger values would require arbitrary-precision arithmetic.

4.4 Potential Improvements

- **Thread pooling:** A fixed pool of reusable workers would amortize creation costs across all N computations.
- **Coarser granularity:** Fewer, larger chunks per worker would improve the computation-to-overhead ratio.
- **Per-value condition variables:** Using one condition variable per $C(n)$ would eliminate spurious wake-ups at the cost of additional code complexity.
- **Big-integer arithmetic:** For large N, using a library such as GMP would make each multiplication expensive enough that the worker-level parallelism yields genuine speed-ups.

References

[1] T. Davis. Catalan numbers. Unpublished manuscript, November 2006. http://www.geometer.org/mathcircles/catalan.pdf.

[2] OEIS Foundation. Sequence A000108: Catalan numbers. *The On-Line Encyclopedia of Integer Sequences*, 2026. https://oeis.org/A000108.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne. *Operating System Concepts*. Wiley, 10th edition, 2018.

[4] M. Kerrisk. *The Linux Programming Interface*. No Starch Press, 2010.